

## Using Llama locally

1. Go to <https://ollama.com/download> to download ollama, this application help you to run various LLM such as Llama, Deepseek, etc locally
2. You can go to <https://ollama.com/search> to search for the model, it's kind like model repository
3. After installation completed, you must be able to run ollama in Command Prompt

```
C:\Users\Asus>ollama
Usage:
  ollama [flags]
  ollama [command]

Available Commands:
  serve      Start ollama
  create     Create a model from a Modelfile
  show      Show information for a model
  run       Run a model
  stop      Stop a running model
  pull      Pull a model from a registry
  push      Push a model to a registry
  list      List models
  ps       List running models
  cp       Copy a model
  rm       Remove a model
  help     Help about any command

Flags:
  -h, --help      help for ollama
  -v, --version   Show version information

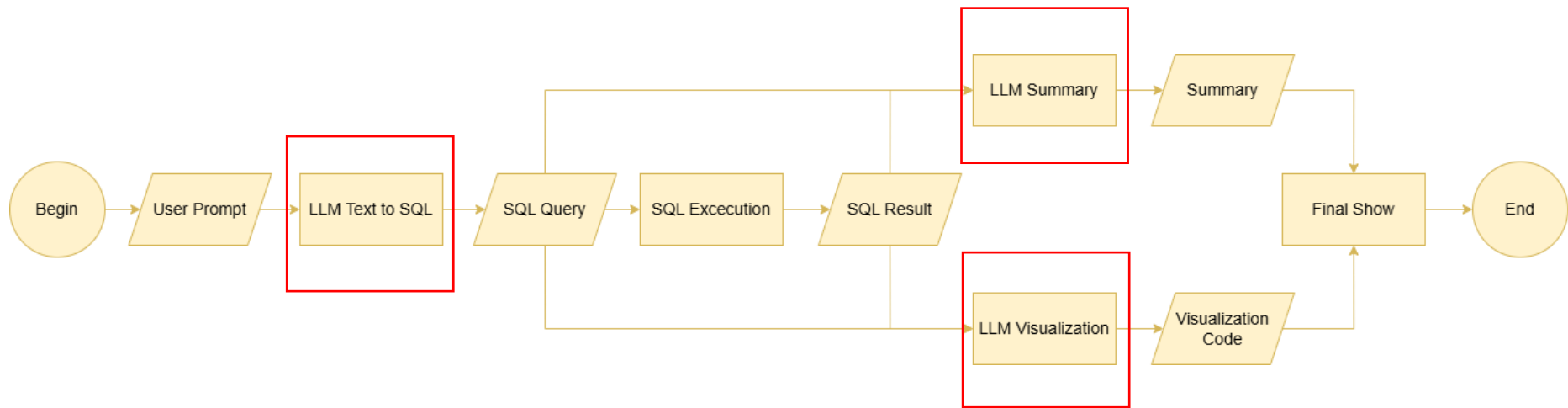
Use "ollama [command] --help" for more information about a command.
```

4. And now you can pull the model and run it locally
5. Now, how do we use it in programming languages?
6. I suggest you to use Langchain Framework, that can be easily used to integrating with any tools

```
sql_llm = OllamaLLM(model="llama3.1:8b")
summary_llm = OllamaLLM(model="llama3.1:8b")

new_prompt = "buatkan saya query sql untuk menampilkan nama dan harga dari tabel produk"
response = summary_llm.invoke(new_prompt)
```

7. Then we can serve in Flask/Django so PHP can call using API



1. There are 3 LLM process, but we can run concurrent on summary and visualization, so the time consumed will be 2 LLM process

## Tutorial Code

### A. Dari sisi PHP

1. Hanya perlu index.html, yang dikotakin sesuaikan dengan ip dan port yang ada di file python

```
<script>
function submitQuery() {
  const userQuery = document.getElementById('user_query').value;
  console.log("asdasd");
  if (!userQuery) {
    alert('Tolong masukkan pertanyaan.');
```

// harus sama dengan python nya

```
    fetch('http://127.0.0.1:1000/generate', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ prompt: userQuery })
    })
    .then(response => response.json())
    .then(data => {
      document.getElementById('output_area').innerHTML = `<strong>
        <strong>Hasil:</strong> <pre>${JSON.stringify(data.result)}</pre>
        <strong>Respon LLM:</strong> <p>${data.response}</p>`;
    })
    .catch(error => {
      document.getElementById('output_area').innerHTML = `<p style=
        console.error('Error:', error);
      });
    });
  }
}
</script>
```

### B. Dari sisi Python

1. Edit variable untuk database dan model LLM, cari **file .env**

```

1 DB_HOST=localhost
2 DB_USER=chatdb
3 DB_PASSWORD=chatdb
4 DB_NAME=wren-sql
5
6 LLM_MODEL_NAME=llama3.1:8b
```

2. Buat virtual env, buka cmd di folder proyek tersebut. Ini bertujuan agar menginstall library yang diperlukan:

```
python -m venv venv
venv\Scripts\activate.bat
pip install -r requirements.txt
```

### 3. Bisa bermain dengan prompt

```
# Function to generate SQL from prompt
def query_database(prompt):
    original_prompt = f"{prompt}. Berikan sql yang lengkap saja tanpa penjelasan. Pastikan sql yang kamu berikan sudah sesuai dengan database dan dapat dijalankan"
    query_with_context = {"question": original_prompt}

# Function to generate a friendly response
def generate_response(prompt, sql_query, sql_result):
    new_prompt = f"Kamu adalah customer service yang ramah. Berdasarkan pertanyaan user {prompt}, didapatkan sql query seperti berikut: {sql_query}. Hasilnya adalah {sql_result}. Buatlah kesimpulan dari data yang diberikan dengan bahasa yang mudah dipahami oleh user."
    response = summary_llm.invoke(new_prompt)
```